Opening · the programme this tool belongs to

Legible: checked before it is handed on

"Everything that comes before code, from requirements and processes to models, decisions and contracts, ***made query-able for humans and AI, and checked before it is handed to a developer or a dev-agent.***"

Legible — query-able by humans and AI

- one identity per entity, in every artefact that names it
- stable identifiers other artefacts can reference
- machine-readable forms beside the human ones
- traceable both ways: requirement → model → test

Checked — gated before hand-off

- a deterministic check on every change, at the desk and in the pipeline
- findings with a severity; a threshold fails the build
- a baseline, so an existing estate adopts it without a clean-up
- a graded score that shows progress

pumllint is the programme's reference gate: it checks PlantUML diagrams, and business processes converted from ARIS by **aris2puml**. The rest of this session is about that gate.

fdurieux.github.io

The map: every artefact before code, the tool or practice that gates it, and the gaps still open.

Is semantic linting of diagrams worth a PoC?

Three questions for this round table

AGREE

1 The problem

Do we recognise it?

- semantic correctness
- common pain points
- analysts & process engineers
- developers & AI tools

→ *a shared problem*

UNDERSTAND

2 The contribution

What does a semantic linter add?

- linting rules
- conventions
- quality checks
- engineering workflows

→ *a shared understanding*

DECIDE

3 The value

Is it worth trying?

- documentation quality
- collaboration
- automation
- delivery lifecycle

→ *a go / no-go for a PoC*

PoC?
go / no-go

Problems to solve, across the delivery pipeline

Continuous Exploration
where intent is formed

- A diagram renders but does not make sense: a typo becomes a phantom participant, a branch is guarded by “sometimes”, one service is described two ways.
- Requirements and stories: their structure is checked, their wording and ambiguity are not.
- Models and decisions: gated for PlantUML, C4 and OpenAPI; open for ArchiMate, SysML, ARIS processes and ADRs.

Continuous Integration
where the checks run

- Code has had quality gates for twenty years; the information it is built from has had none.
- A gap found here is found late: someone has already built on it.
- AI builds from the design at near-zero cost and fills gaps with guesses: from Level 1 sequence diagrams ~1 in 3 intended behaviours fail. [measured]

Continuous Deployment
mostly no claim

- Deploy, verify, monitor: nothing upstream is checked here, and this session claims nothing for them.
- Respond: responders inherit the models, if they are current and if they consult them. [hypothesis]

Release on Demand
governance and learning

- No evidence that the design documentation is current when an auditor asks.
- No trend of documentation health to steer on.
- A process design never compared with what actually happens. [planned]

Shift-left is a split, not a point: the artefacts are written in Continuous Exploration; the gates on them run in Continuous Integration.

What we mean by semantic correctness

Logical correctness — 20%

Is it logically sound?

A block that never closes; a state machine with no starting point.

Completeness · *gaps* — 30%

Is everything that should be there actually there?

A process with no end; an error path nobody drew.

Ambiguity · *ambiguities* — 25%

Could two readers take it two different ways?

Arrows with no label; conditions like "sometimes"; "TBD".

Consistency — 15%

One convention, here and across diagrams?

One thing with two names, kinds or stereotypes.

Traceability — 5%

Can you find it, cite it, and tell who owns it?

No title, no owner, no link to the requirement.

Readability — 5%

Is it small enough to take in?

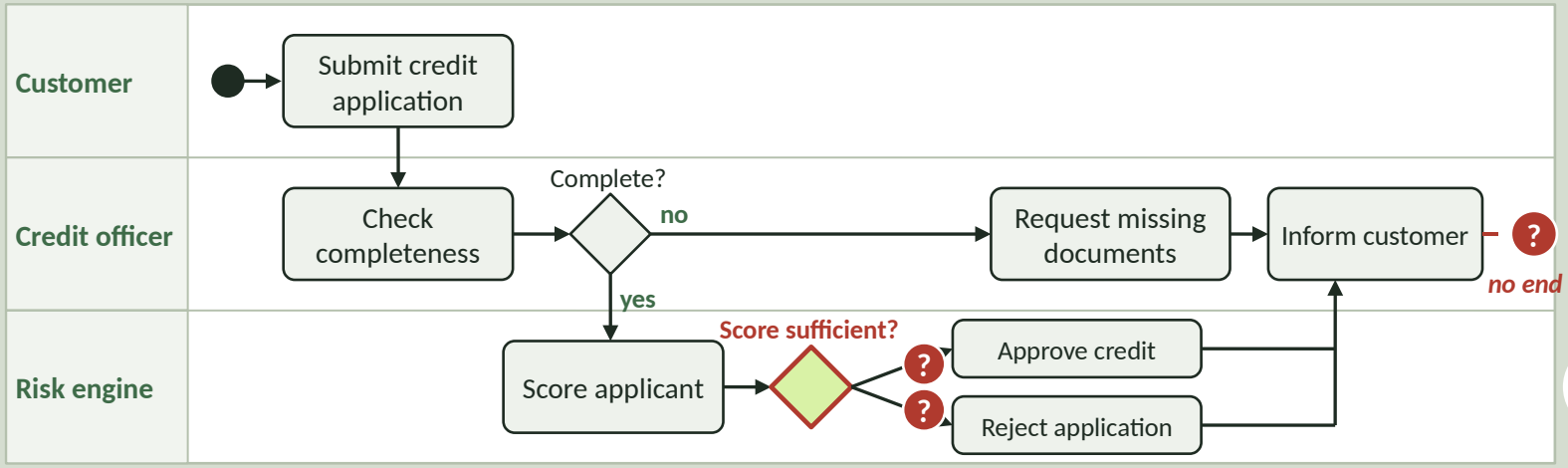
Too many lifelines or messages on one canvas.

Underneath: syntax Does it render? Solved: PlantUML and its IDE plugins already check this.

① The problem · a credit flow: one gap, passed on to the next diagram

Both render. Which branch approves?

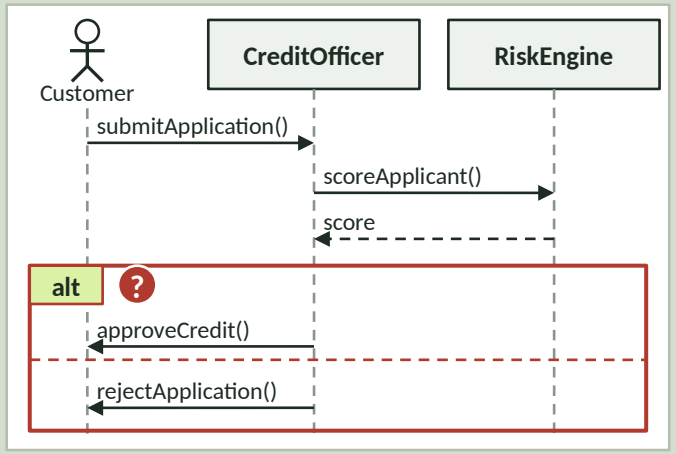
`credit-application.puml` *business process* — ✓ renders




```
if (Score sufficient?) then … else … :Inform customer; @enduml   ← no stop
```

`credit-decision.puml` ✓ renders

derived from the process




```
alt   ← no condition
```

pumllint, linting both files

ACT003 · minor
credit-application: the decision's branches are not labelled

ACT002 · major
credit-application: the process never ends; the build fails

SEQ007 · minor
credit-decision: alt without a condition; the guess, carried down

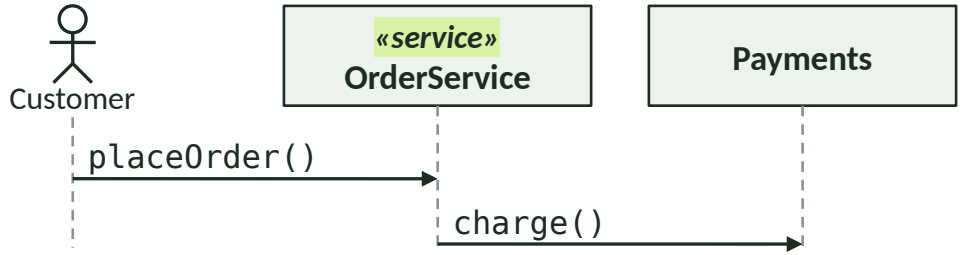
① The problem · each file is valid, together they contradict

Both render. Which one is right?

checkout.puml

✓ renders

```
@startuml checkout
actor Customer
participant OrderService <<service>>
participant Payments
Customer -> OrderService : placeOrder()
OrderService -> Payments : charge()
@enduml
```

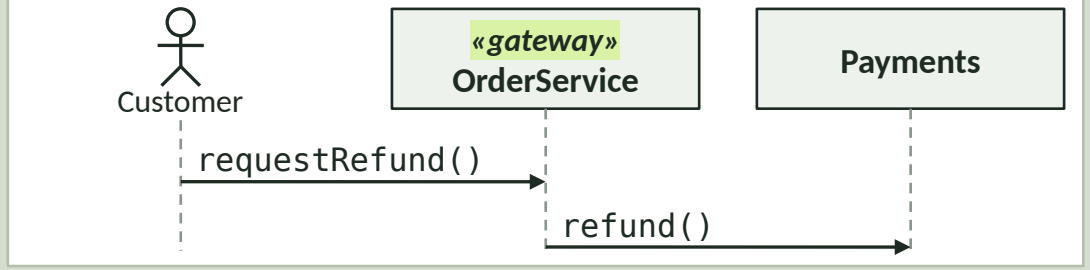


≠

refund.puml

✓ renders

```
@startuml refund
actor Customer
participant OrderService <<gateway>>
participant Payments
Customer -> OrderService : requestRefund()
OrderService -> Payments : refund()
@enduml
```

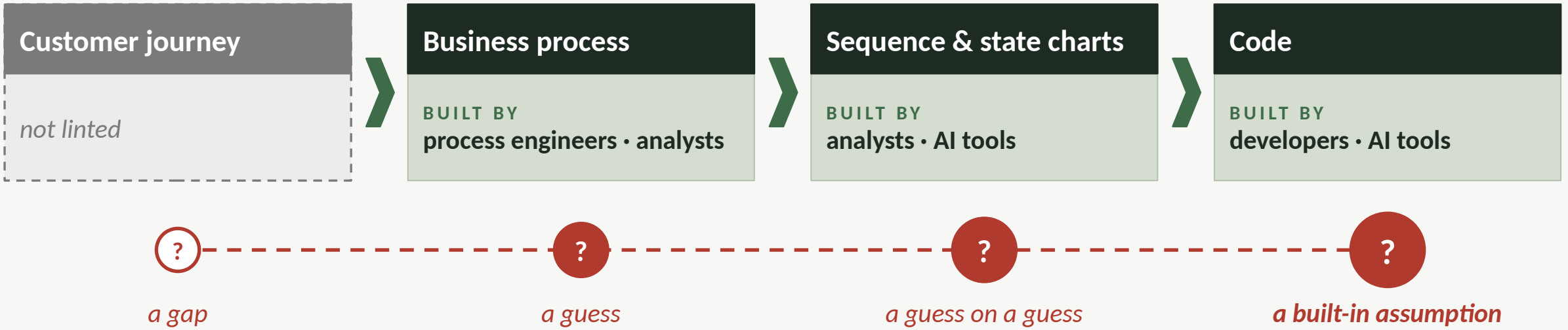


pumllint, linting both files together

XD002 · consistency · OrderService is «service» in checkout.puml and «gateway» in refund.puml: one entity, one stereotype

① The problem · humans and AI tools build on what the diagram says, gaps and ambiguities included

Semantic quality: why it matters now



Hallucinations are not exclusive to AI

Every reader fills the gaps: an analyst, process engineer, developer or AI tool turns what is unclear into a guess.

Diagrams form a chain

Each level is the foundation for the next, gaps included. One gap upstream becomes a guess at every step below.

AI raises the stakes

AI tools read and write diagrams at scale: more volume, the same gaps, and they never ask.

② The contribution · the gates that exist, and where pumllint sits

Tooling across the delivery pipeline

pumllint no gate yet

Continuous Exploration	Continuous Integration	Continuous Deployment	Release on Demand
requirements and prose **vale, markdownlint**	source code **ESLint, ruff, SonarQube**	infrastructure as code **tflint, checkov**	licences · SBOMs **REUSE lint · NTIA checker**
architecture diagrams (PlantUML) **pumllint**	commits · containers **commitlint · hadolint**	Kubernetes manifests **kubeconform**	design-governance evidence **pumllint badge and HTML report**
ARIS processes (EPC) **aris2puml, then pumllint**	CI workflows **actionlint**	policy as code **OPA conftest**	**planned** process design vs event logs
API contracts **Spectral**	diagrams, on every change **pumllint: CI and pre-commit**		
BPMN processes · C4 models **bpmnlint · Structurizr validate**	**open gap** diagram-versus-code drift		
open gap ADRs, ArchiMate, SysML, stories			

Downstream artefacts are gated; the upstream one mostly is not. pumllint brings the pattern the pipeline already trusts to the design.

② The contribution · a semantic static analyser for PlantUML models

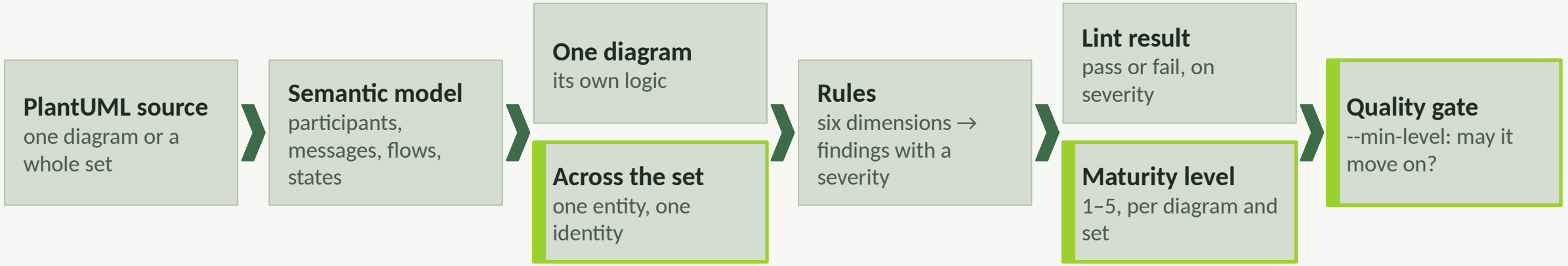
From linter to static model analysis

A CONVENTIONAL LINTER

Text › **Pattern rule** › **Warning**

e.g. "line too long", "unused variable": it reads the text, not what it means

PUMLLINT

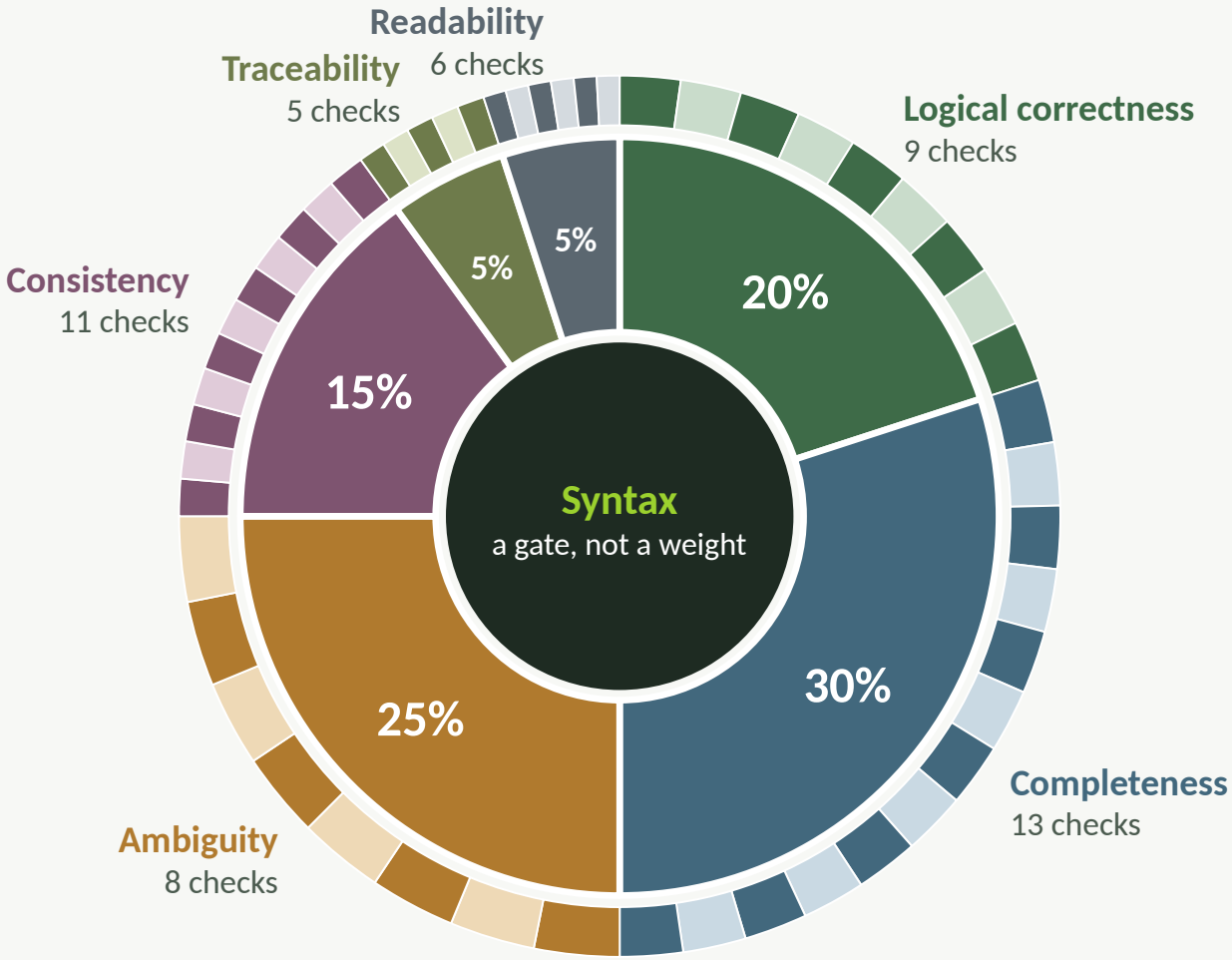


DETECT | MEASURE | GOVERN

What a conventional linter does not do. The rules stay the source of truth; the level adds them up, the gate applies your policy.

② **The contribution · the checks behind the level, drawn as a wheel**

What the score is made of



The size of a segment is its weight in the score. Completeness and ambiguity count most, because those are the gaps the next reader fills with a guess.

Each finding counts against its dimension by severity: blocker 10, critical 8, major 5, minor 2, info 0.5.

The outer ring has one wedge per check: 52 in all. 9 run only with the code-generation profile; 5 stay off until a team sets its own convention.

Syntax sits in the middle as a gate: a diagram that does not render is Level 1, whatever the rest says.

Every check, with what it looks for: slides 24 to 27 (appendix). Read from pumllint 0.35.0's rule catalogue when this deck was built.

② **The contribution · not always an error, always a reason to guess**

Code smells, model smells

Code smell · SonarQube

Something in the code that makes it harder to understand, change or trust; not always a bug.
e.g. too complex · unreachable branch · unclosed resource

≈

Model smell · pumllint

Something in the diagram that makes it harder to interpret, implement or trust: **a reason for the next reader to guess.**

Dimension	Diagram	Model smell	Rule	Severity
Logical correctness	sequence	A participant sends messages but was never declared	`SEQ001`	**critical**
Completeness	business process	The flow never reaches an end	`ACT002`	**major**
Ambiguity	state chart	A transition with no label: what triggers it?	`STA003`	**minor**
Consistency	across the set	One participant is a service here, a database there	`XD001`	**major**
Traceability	any	No link to the requirement or decision it implements *	`GEN007`	**minor**
Readability	any	Too many elements on one canvas (over 60)	`GEN009`	**minor**

** checked once your requirement-tag format is configured*

② **The contribution · every finding has a severity; every diagram gets a level**

Detect, measure, govern

1 DETECT findings

Severity	Meaning	Effect
Blocker	Unusable as a specification	**Fails the build · max Level 2**
Critical	Broken structure	**Fails the build · blocks Level 5**
Major	Breaks a mandatory standard	**Fails the build · blocks Level 5**
Minor	Breaks a recommended convention	Reported · lowers the score
Info	Advisory	Reported · lowers the score slightly

2 MEASURE maturity

Level	Name	Read it as
5	**Method-complete**	Every dimension strong, no majors: preconditions for AI code generation met (codegen profile only)
4	**Precise**	Complete and unambiguous: ready for hand-off
3	**Disciplined**	No blockers: ready for review, not yet for hand-off
2	**Structured**	Coherent enough to work on, although it may still contain blockers
1	**Sketchy**	Don't build on it

3 GOVERN

The team sets the bar, e.g. Level 2 everywhere, Level 4 before hand-off: a diagram below it does not move on.
Business processes: gate on findings, not on the level; no ambiguity rule covers activity diagrams yet.

② **The contribution · the level predicts what happens downstream**

Does the level matter? Measured: yes

Sequence diagrams at Level 1

~1 in 3

intended behaviours failed when the generated code ran

vs

Sequence diagrams at Level 2 and up

~1 in 10

intended behaviours failed when the generated code ran

Not an opinion: AI tools generated the code, and it was run against acceptance tests written beforehand.

Robust: held across three generators from two vendors and three prompt styles.

No prompt fixes it: better prompting helped untidy diagrams, never the Level 1 ones; it cannot add guards and failure paths the diagram never had.

Scope: sequence diagrams and AI-generated code only. **Expected, not measured:** less rework and a shorter lead time; a PoC is how we would find out.

Source: github.com/fdurieux/pumllint · EVIDENCE.md, "What the product may claim" · in plain English: docs/evidence-explained.md, "Verdict 1"

③ In practice · one business case, four linked diagrams, drawn twice

The demo: a credit application

requirements.txt	credit-application	credit-decision	credit-scoring	application-status
CRD-101 … CRD-105 what must be designed	→ **business process** who does what, in which order	→ **sequence diagram** the decision, derived from the process	→ **sequence diagram** one step in detail	→ **state chart** the application's life

Linked like a real set: the same names (Customer, CreditOfficer, RiskEngine) and the same requirement IDs in every diagram, so pumllint can check the set as a whole and trace each requirement to the diagrams that draw it.

Compliant	✔️ **Gate passes** no findings · every diagram at Level 4 · 5 of 5 requirements drawn
Non-compliant	✖️ **Gate fails** 10 findings (1 critical, 2 major, 7 minor) · still Level 4 · 4 of 5 requirements drawn, 1 unknown ID cited

Every one of the eight diagrams renders.

③ In practice · the compliant versions pass; here is what the others get wrong

Valid or not, and why

Diagram	What is wrong in the non-compliant version	Rule · severity	What the next reader has to guess
Business process `credit-application.puml`	The score decision does not name its branches, and the flow never reaches an end.	`ACT003 ×2 · minor` `ACT002 · major`	*Which branch approves? When is an application finished?*
Decision `credit-decision.puml`	An “alt” with no condition; a typo, “Custmer”, quietly adds a second customer.	`SEQ007 · minor` `SEQ001 · critical` `XD002 · minor`	*When does each branch apply? Who is the second customer?*
Scoring `credit-scoring.puml`	RiskEngine is a <<gateway>> here and a <<service>> in the decision diagram.	`XD002 · minor`	*Does the risk engine hold the logic, or only pass calls on?*
Status `application-status.puml`	Nothing says what makes an application Approved; nothing leads to Withdrawn; no owner is named.	`STA003 · minor` `STA002 · major` `GEN006 · minor`	*What triggers approval? Can an application be withdrawn? Whom do I ask?*
Requirements	CRD-105 (“the customer is informed”) is drawn nowhere; CRD-106 is cited but does not exist.	`trace`	*Is informing the customer designed at all? Typo, or a missing requirement?*

Every non-compliant diagram still scores Level 4. The level says how complete and precise a drawing is overall; the gate looks at how serious each finding is. One critical or major finding stops the hand-off.

③ In practice · the same set, live, on this laptop

Live demo: localhost:8000

START IT

```
$ cd sandbox/credit-application
$ docker compose up
  → open http://localhost:8000
```

Two containers on this computer: a PlantUML server that draws the diagrams, and a page that shows pumllint's findings next to each one. The first start downloads them; after that it runs offline.

Run it yourself
sandbox/credit-application in the pumllint repository: the diagrams, the README, and a 15-minute walkthrough. Without Docker, pip install pumllint gives the same findings, without the pictures.

FIFTEEN MINUTES, FIVE STEPS

1. **Every diagram renders**
   Which of these eight would you sign off?
2. **The compliant set**
   Gate passes, Level 4, every requirement drawn.
3. **The non-compliant set**
   Read the finding, then point at the picture.
4. **Fix one, live**
   Name the branches, add "stop", save, reload: the findings disappear.
5. **Levels, gates, requirements**
   Why Level 4 still fails, and the requirement drawn nowhere.

③ **The value · from advice to gate, step by step, without a big-bang cleanup**

Diagram quality, built into the workflow

On your own machine (with or without Git)

When	How
While writing	In any editor that supports LSP: one language server, no editor-specific plugins
On demand	From the command line: check a whole folder at once, with each diagram's findings and maturity level, plus an HTML maturity report

In the delivery pipeline (diagrams stored in Git)

When	How
On every change	A CI gate checks every changed diagram, whoever authored it; the severity of its findings decides whether the build passes
From existing tools	Business processes modelled in ARIS are converted to PlantUML and pass the same gate
Over time	Dashboards (SonarQube), a maturity report, and a badge per repository showing its weakest diagram's level

Step 1 · ADVISE
report only; nothing fails

Step 2 · RATCHET
record today's levels; fail only when a diagram gets worse

Step 3 · FLOOR
require a minimum level, e.g. Level 2 everywhere, Level 4 before hand-off

Questions

1. What would a PoC need to show to convince you?
2. Where should the gate sit, and which level for which diagrams?

Closing · what we heard, and what we decide

A PoC on semantic linting: go or no-go?

WHAT WE DISCUSSED

1 **The problem**	Diagrams render, but not all of them make sense; every reader downstream fills the gaps.
2 **The contribution**	A semantic linter finds model smells, measures maturity and gates the hand-off.
3 **The value**	Checked in the editor and in CI; start with advice, no big-bang cleanup.

THE VOTE



GO
run the PoC



NO-GO
not now

No-go? Ask what would change your mind, and note it.

IF GO, WE AGREE BEFORE WE LEAVE

Scope	Success criteria	Owner & timebox	Next check-in
which team, which diagrams …	*what must the PoC show?* …	*who leads, how many weeks* …	*when we look at the results* …

Closing · where to read more

pumllint on GitHub

github.com/fdurieux/pumllint

Open source (GPL-3.0) · pip install pumllint · Python 3.11 or later, no dependencies

START HERE

Legible
the programme: every artefact before code, its gate, its gaps

README
what pumllint does, install, quick start

Guides by audience
which document to read for your role

The demo sandbox
this session's credit application, to run yourself

WHY IT MATTERS

The case for pumllint
the management case: cost and payoff

pumllint in the SDLC
where the value lands, claim by claim

The evidence, explained
the measured result, from scratch

Where tooling pays
the delivery-tooling landscape

USING IT

Findings and scores
reading findings and maturity levels

The 360° rule map
every check, by dimension, with examples

Linting business processes
ARIS processes as activity diagrams

Setup and CI
editor, pipeline, ratchet, SonarQube

Appendix · pumllint for the PoC · what we decide

A PoC on semantic linting: go or no-go?

WHY, IN THREE LINES

1 **The problem**	Diagrams render, but not all make sense; every reader downstream, human or AI, fills the gaps.
2 **The contribution**	A semantic linter finds model smells, gives each diagram a maturity level and gates the hand-off, in the editor and in CI.
3 **Measured**	AI code from Level 1 sequence diagrams fails ~1 in 3 intended behaviours; from Level 2 up, ~1 in 10.

YOUR TEAM

Team / squad	…
Team contact	*name, role*
Raised by	*round-table attendee*
Date	…

Your answer ☐ **Go** ☐ **No-go** ☐ **Not yet** **What would change our mind?** …

IF GO, WE AGREE ON

Scope	Success criteria	Owner & timebox	Next check-in
which team, which diagrams	*what must the PoC show?*	*who leads, how many weeks*	*when you look at the results*

Return to: **[presenter name]** · **[email]** · by **[reply-by date]**

Getting started: the three pumllint slides that follow

Appendix · pumllint for the PoC · install and first run

Get started in ten minutes

SIX COMMANDS

```
$ pip install pumllint            # or pipx / uv tool install
$ pumllint diagrams/              # findings, with severity
$ pumllint score diagrams/        # maturity level per diagram
$ pumllint score diagrams/ -f html -o report.html
$ pumllint --list-rules           # everything it checks
$ pumllint lsp                    # findings in your editor
```

WHAT YOU GET BACK

Findings
rule, severity and line, e.g. ACT002 · major

A maturity level
per diagram and per set, with what it takes to reach the next one

An HTML report
one self-contained page, to share or attach to a review

Exit codes for CI
0 clean · 1 findings at or above --fail-on (default major) · 2 usage error

Python 3.11 or later · no runtime dependencies (PyYAML only for a YAML config). Read more in the repository: README · docs/findings-and-scores.md · docs/rule-map.md · docs/business-processes.md

Fit it to your team

Configure

pumllint.yaml (or .toml / .json) is picked up automatically:

```
rules:
  unnamed-diagram: false
  participant-naming:
    severity: major
  requirement-link:
    pattern: "REQ-\d+"
```

One exception in one diagram: `' pumllint: disable=SEQ006`

Automate

GitHub Actions, pinned to a release:

```
- uses: fdurieux/pumllint@v0.35.0
  with:
    command: score
    paths: docs/diagrams
    baseline: maturity.json
    min-level: "2"
```

Or per commit: the pre-commit hooks pumllint and pumllint-score.

Connect

SonarQube
-f sonar writes SonarQube's import format: findings on the same dashboard and in pull requests.

ARIS
the separate aris2puml tool converts processes to PlantUML; they pass the same gate.

Editors
pumllint lsp in any editor that supports LSP; no editor-specific plugins.

Step 1 · ADVISE report only, no gate → **Step 2 · RATCHET** `--baseline maturity.json` → **Step 3 · FLOOR** `--min-level 2`

Appendix · pumllint for the PoC · a suggested plan

A four-week PoC, and what to measure

Week 1 · Baseline	Week 2 · Advise	Week 3 · Ratchet	Week 4 · Review
One team and its diagrams. Score them, publish the HTML report, record maturity.json.	pumllint lsp in the editors; authors fix as they go. Nothing fails.	In CI with --baseline: no diagram may get worse.	Compare with the baseline; agree the floor and the next step.

What to measure

- Findings per dimension, week 1 against week 4
- Sequence and state diagrams at Level 4; processes without a major finding
- Time to fix a finding, as the authors report it
- Noise: findings the team disagrees with, and the config changes they lead to
- Feedback from authors and reviewers
- Rework found downstream: the expected benefit, measured here for the first time

Success criteria
Agreed on the closing slide: what must the PoC show? Write them here before week 1, so week 4 compares against them.

A proposal to adapt: the length, the team and the measures are the PoC owner's call.

Appendix · every check behind the score (1 of 4)

Completeness, 30%

Check	Name	What it looks for	Severity	Diagrams	When
Completeness 13 checks · 30% of the score					
SEQ002	unused-participant	Declared participant never appears in a message	minor	sequence	
SEQ007	unlabelled-block-condition	Grouping block has no condition/label text	minor	sequence	
SEQ010	explicit-participant-order	Participant introduced by first use instead of an explicit declaration	info	sequence	if set
SEQ101	codegen-implicit-participant	Participants must be explicitly declared, not created on first use	blocker	sequence	codegen
SEQ102	codegen-untyped-participant	Participant declarations must carry a role type or stereotype	major	sequence	codegen
SEQ104	codegen-missing-return	Synchronous calls must have an explicit return message	major	sequence	codegen
SEQ107	codegen-missing-failure-path	Calls to <<external>>/database/queue participants need an enclosing error fragment, by label vocabulary (the 'failure_keywords' lexicon)	major	sequence	codegen
ACT001	missing-start	Activity diagram has actions but no 'start' node	major	activity	
ACT002	missing-stop	Activity diagram has no 'stop'/'end' terminal node	major	activity	
ACT003	unlabelled-decision-branch	Decision branch ('then'/'else') or loop outcome ('is'/'not'/'endwhile') has no (label)	minor	activity	
CLS002	association-multiplicity	Association does not declare multiplicities on both ends	major	class	
STA002	unreachable-state	State has no incoming transition	major	state	
UC001	orphan-actor-or-usecase	Actor or use case has no relationships	major	use case	

When: "codegen" runs only with the code-generation profile; "if set" stays off until a team configures its convention. From pumllint 0.35.0's rule catalogue.

Appendix · every check behind the score (2 of 4)

Ambiguity, 25% · Traceability, 5%

Check	Name	What it looks for	Severity	Diagrams	When
Ambiguity 8 checks · 25% of the score					
SEQ005	unlabelled-message	Message has no label text after the colon	minor	sequence	
SEQ006	no-self-message	Self-message; internal logic belongs in a note	minor	sequence	
SEQ103	codegen-prose-message	Call messages must be signature-shaped, name(args) — the accepted shape is the 'pattern' option	blocker	sequence	codegen
SEQ105	codegen-vague-guard	alt/opt/loop guards must be present and not a known vague phrase (the 'vague_terms' lexicon)	blocker	sequence	codegen
SEQ106	codegen-elision-marker	No placeholder/elision markers (..., TBD, etc.) in labels, guards or notes	blocker	sequence	codegen
SEQ109	codegen-uninformative-reply	Replies must use reply arrows (-->) with a label that is not a known non-informative word (the 'non_informative' lexicon)	minor	sequence	codegen
CLS003	unlabelled-association	Plain association has no role/verb label	minor	class	
STA003	unlabelled-transition	State-to-state transition has no event/guard/action label	minor	state	
Traceability 5 checks · 5% of the score					
GEN001	missing-title	Diagram has no title	minor	all	
GEN002	unnamed-diagram	@startuml has no diagram name	info	all	
GEN006	owner-tag	Diagram declares no ownership tag (dormant until a pattern is configured)	minor	all	if set
GEN007	requirement-link	Diagram references no requirement/ADR (dormant until a pattern is configured)	minor	all	if set
GEN010	duplicate-diagram-name	Two or more diagrams in one file share a @startuml name	minor	all	

When: "codegen" runs only with the code-generation profile; "if set" stays off until a team configures its convention. From pumllint 0.35.0's rule catalogue.

Logical correctness, 20% · Readability, 5%

Check	Name	What it looks for	Severity	Diagrams	When
Logical correctness 9 checks · 20% of the score					
SEQ001	undeclared-participant	Participant used in a message but not declared with participant/actor/...	critical	sequence	
SEQ003	unbalanced-activation	activate/deactivate pairs do not balance for a lifeline	major	sequence	
SEQ004	unterminated-block	Grouping block (alt/opt/loop/par/.../box) is never closed with 'end'	critical	sequence	
SEQ009	unpaired-return	Dashed return message does not pair with a preceding call	minor	sequence	
SEQ108	codegen-activation-lifecycle	activate/deactivate must pair as a well-formed stack per lifeline	major	sequence	codegen
ACT004	unterminated-construct	Activity construct (if/while/repeat/fork/switch/partition) is never closed	critical	activity	
CLS004	inheritance-cycle	Generalization/realization hierarchy contains a cycle	major	class	
STA001	single-initial-state	State machine must have exactly one top-level '[*] -->' initial transition	blocker	state	
UC003	include-extend-direction	<<include>>/<<extend>> arrow points the wrong way or involves an actor	minor	use case	
Readability 6 checks · 5% of the score					
GEN005	max-participants	Diagram has more participants than the configured maximum	minor	sequence, use case	
GEN008	note-density	Too many notes for the diagram's size — structure narrated instead of modelled	minor	all	
GEN009	max-elements	Diagram has more semantic elements than the configured maximum	minor	all	
SEQ008	fragment-nesting-depth	Combined fragments nested beyond the configured maximum depth	minor	sequence	
SEQ011	max-messages	Diagram has more messages than the configured maximum	minor	sequence	
CLS005	max-members-per-class	Class has more members than the configured maximum	minor	class	

When: "codegen" runs only with the code-generation profile; "if set" stays off until a team configures its convention. From pumllint 0.35.0's rule catalogue.

Consistency, 15%

Check	Name	What it looks for	Severity	Diagrams	When
Consistency 11 checks · 15% of the score					
GEN003	inline-skinparam	Inline skinparam found; prefer a central !include theme	minor	all	
GEN004	participant-naming	Participant name violates naming convention	minor	sequence	
XD001	conflicting-participant-kind	Same participant declared with different kinds across diagrams	major	sequence	
XD002	conflicting-participant-stereotype	Same participant carries different stereotypes across diagrams	minor	sequence	
XD003	participant-name-case-collision	Participant names differing only by case across diagrams	minor	sequence	
XD004	cross-type-name-collision	Entity names differing only by case across diagram types	minor	all	
XD005	cross-type-stereotype-conflict	Entity carries different stereotypes across diagram types	minor	all	
ACT005	swimlane-naming	Swimlane name violates the naming convention	minor	activity	
ACT006	verb-first-activity	Activity name is not phrased verb-first	minor	activity	if set
CLS001	class-naming	Class or member name violates the naming convention	minor	class	
UC002	usecase-actor-naming	Use case is not phrased as a verb–object phrase	minor	use case	if set

When: "codegen" runs only with the code-generation profile; "if set" stays off until a team configures its convention. From pumllint 0.35.0's rule catalogue.